

Hashing 2

Question: Given array of elements and k , check if there exists a pair (i, j) such that $arr[i] + arr[j] = k$ and $i \neq j$

0 1 2 3 4 5 6 7 8
 $arr = [8, 9, 1, -2, 4, 5, 11, -6, 4]$

$k = 6$ - True

$k = 22$ - False

$k = 8$ - True

Brute Force Approach:

- Iterate over all pairs
- Check if a pair exists whose sum = k .

$$T.C = O(N^2) \quad S.C = O(1)$$

Idea: Using Hashset.

0 1 2 3 4 5 6 7 8

$arr = [8, 9, 1, -2, 4, 5, 11, -6, 4]$

$k = 8$

$arr[i]$

$arr[j]$

$HS = \{8, 9, 1, -2, 4, 5, 11, -6\}$

8

0

9

-1

1

7

-2

10

4

4

$\rightarrow \text{True}$

$T.C = O(N)$ $S.C = O(N)$

Edge Case: $arr = [5, 7, 9, 2, 3]$

$k = 4$

$arr[i]$

$arr[j]$

5

-1

7

-3

9

-5

2

2

$\rightarrow \text{True}$

With the fix:

$arr = [5, 7, 9, 2, 3, -1]$

$k = 4$

$arr[i]$ $arr[j]$

$HS = \{5, 7, 9, 2, 3\}$

5

-1

7

-3

9

-5

2

2

→ False

3

1

-1

5

→ True

First check if element is present in HS and then add it to get the correct result.

Using Hashmap:

$arr = [5, 7, 9, 2, 5]$

$k = 4$

$HM = \{5: 1, 7:$

$1, 9: 1,$

$2: 1, \}$

$arr[i]$

$arr[j]$

5

-1

7

-3

9

-5

2

2

→ False

5

-1

```
func checkPairSum(int[] arr, int k) {  
    hm = {}  
    for (int i = 0; i < arr.length; i++) {  
        co_pair = k - arr[i];  
        if (hm.search(co_pair)) return true;  
        if (hm.search(arr[i])) hm[arr[i]]++;  
        else hm.insert(arr[i], 1);  
    }  
    return false;  
}
```

T.C = $O(N)$ S.C = $O(N)$

Question: Given an array of elements, count the number of pairs such that $\text{arr}[i] + \text{arr}[j] = k$ and $i \neq j$

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ \text{arr} = [3, 5, 1, 2, 1, 2] & k=3 \end{array}$$

$$\begin{aligned} \text{O/P} &= (2, 3) (3, 4) (4, 5) (2, 5) \quad i < j \\ &= 4 \end{aligned}$$

<u>arr[i]</u>	<u>Co-pair</u>	<u>Ans</u>	HM = { 3 : 1, 5 : 1, 1 : 1, 2 : 1 }
3	0		
5	-2		
1	2		
2	1	+1	
1	2	+1	
2	1	+2	

```

func countPairs (int[] arr, int k) {
    hm = {};
    ans = 0;
    for (int i = 0; i < arr.length; i++) {
        co-pair = k - arr[i];
        if (hm.search(co-pair)) ans += hm[co-pair];
        if (hm.search(arr[i])) hm[arr[i]]++;
        else hm.insert(arr[i], 1);
    }
    return ans; // For i < j, else ans * 2;
}

```

<u>arr[i]</u>	<u>co-pair</u>	<u>ans</u>	hm = { 3 : 1,
3	0	0	5 : 1,
5	-2	0	1 : 1, 2
1	2	0	2 : 1, 2
2	1	0 + 1	
1	2	1 + 1	
2	1	2 + 2	

T.C = $O(N)$

S.C = $O(N)$

$arr = [3, 5, 1, 2, 1, 2]$
 $k = 3$

Question: Given an array of elements, check if there exists a subarray with $\text{sum} = k$.

0 1 2 3 4 5 6 7 8
arr = [2, 3, 9, -4, 1, 5, 6, 2, 5]

$k = 11 \rightarrow \text{True}$

$k = 10 \rightarrow \text{True}$

$k = 15 \rightarrow \text{True}$

Brute force Approach:

- Go over each subarray sum
- Check if $\text{sum} == k$, return true.

$$T.C = O(N^2)$$

$$S.C = O(1)$$

Quiz: $A = [5, 10, 20, 100, 105]$ $k = 110$
false

$$PF[i] - PF[j] = k \Rightarrow PF[j] = PF[i] - k$$

Optimised:

↳ Co-pair

arr = [2, 3, 9, -4, 1, 5, 6, 2, 5]

PFSum = [2, 5, 14, 10, 11, 16, 22, 24, 29]

PF[i] Co-pair $\rightarrow$ PF[i] - k k = 15 Bank Balance

2 -13 HM = { 2: 1, 1st Jan $\rightarrow$ 5 L

5 -10 5: 1, 1st April $\rightarrow$ 8 L

14 -1 14: 1,

...

29 14 $\rightarrow$ True

func subarraySumK(int[] arr, int k) {

hm = {}

PF[N] // TODO

for (int i = 0; i < PF.length; i++) {

co-pair = PF[i] - k;

if (hm.search(co-pair)) return true;

if (hm.search(arr[i])) hm[arr[i]]++;

else hm.insert(arr[i], 1);

}

return false;

T.C = O(N)

S.C = O(N)

}

Edge case: $arr = [2, 3, 9, -4, 1]$ $k = 11$

$PFSum = [2, 5, 14, 10, 11]$

$\rightarrow co_pair = 0$

To fix: Add 0 initially into H

↓
Not existing in
the HM.

↓
false

```
func subarraySumK (int[] arr, int k) {
```

```
    hm = {0:1}
```

```
    PF[N] // TODO
```

```
    for (int i = 0; i < PF.length; i++) {
```

```
        co_pair = PF[i] - k;
```

```
        if (hm.search(co_pair)) return true;
```

```
        if (hm.search(arr[i])) hm[arr[i]]++;
```

```
        else hm.insert(arr[i], 1);
```

```
    }
```

```
    return false;
```

```
}
```

T.C = $O(N)$

S.C = $O(N)$

Question: Distinct elements in every window of size k .

$arr = [1, 2, 1, 3, 4, 2, 3]$ $k = 4$

Idea 1:

Optimised Idea: